# Lecture 29 — Frontend Performance

Jeff Zarnett
jzarnett@uwaterloo.ca

Department of Electrical and Computer Engineering
University of Waterloo

November 11, 2026

# Feelings and Facts

Most topics of the course focus on how to actually improve the execution speed of the work to be done.

Frontend performance is unlike those kinds of improvements, because the user's feelings matter.

If they *perceive* the page as fast, that's as good as, or better than, the page actually being faster.

# What Helps Perception

1. Acknowledgement of actions
2. Loading spinners or progress bars
3. Make the user *think* it's done, even when it's not.



# Thresholds

Let’s talk about the thresholds that matter to users:

1. Under 0.1 seconds is “instant”.
2. Between 1 and 10 seconds: use spinner
3. After 10 seconds, the user is impatient...

Notify them? Send the result later?

# Motivation

Aside from the correctness issues, there's also the risk that customers just... stop using your app or abandon whatever they were doing.

If it doesn't show up in voice of customer, is it a problem or do they just quit?

# Taking Too Long

Study: users start abandoning the video load if it doesn't start in two seconds.

Website speed: 4 s is perceived as long; 400 ms is correlated with lower usage.
Tolerance depends on internet connection type.

Patience scales with what they asked for (e.g., GO Schedule vs Quest Report).

# Monetary Motivation




Study: 0.1 second improvement in 15 retail brands' mobile website resulted in 8.4% more conversions and 9.2% higher order value.

# Is It Done?



# Trickery of Timing

We'll also exclude from discussion things that look like only loading the first *n* rows and then loading the rest when the user scrolls down.

What about showing the partial page as soon as we can?

Loading the shell helps a lot.

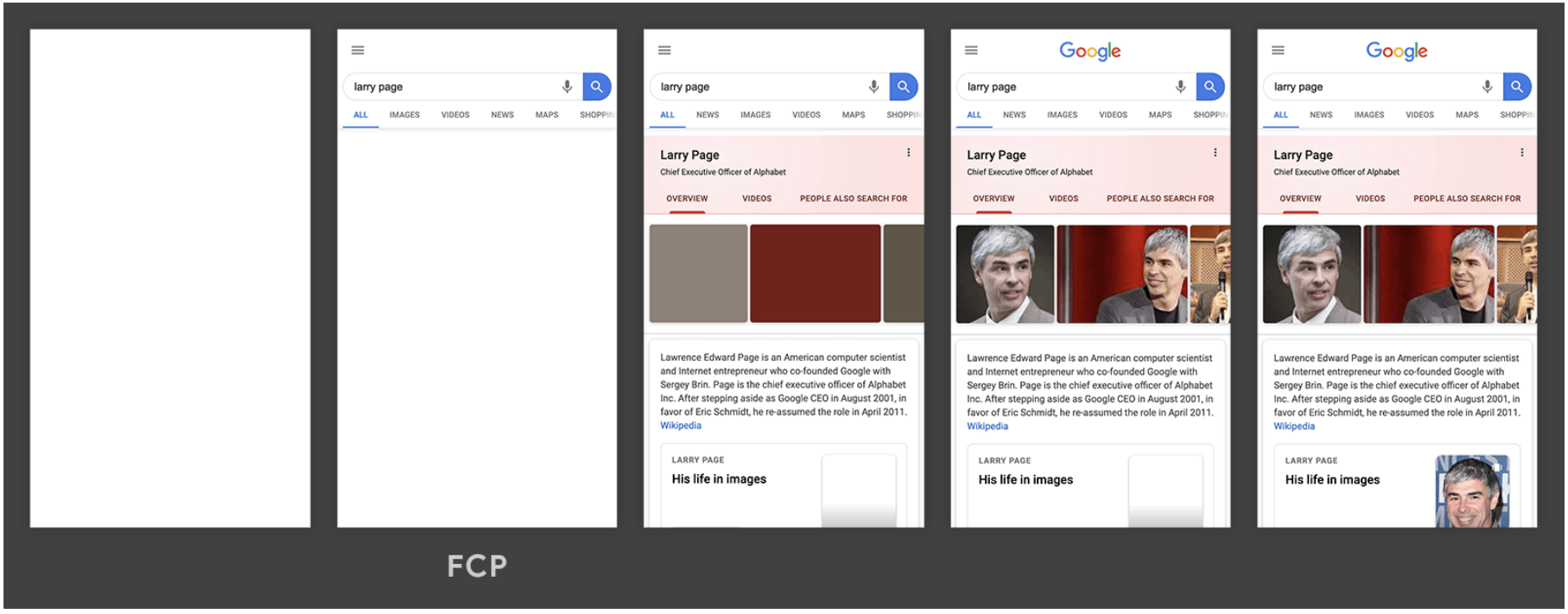
# Definitions

FCP: First Contentful Paint

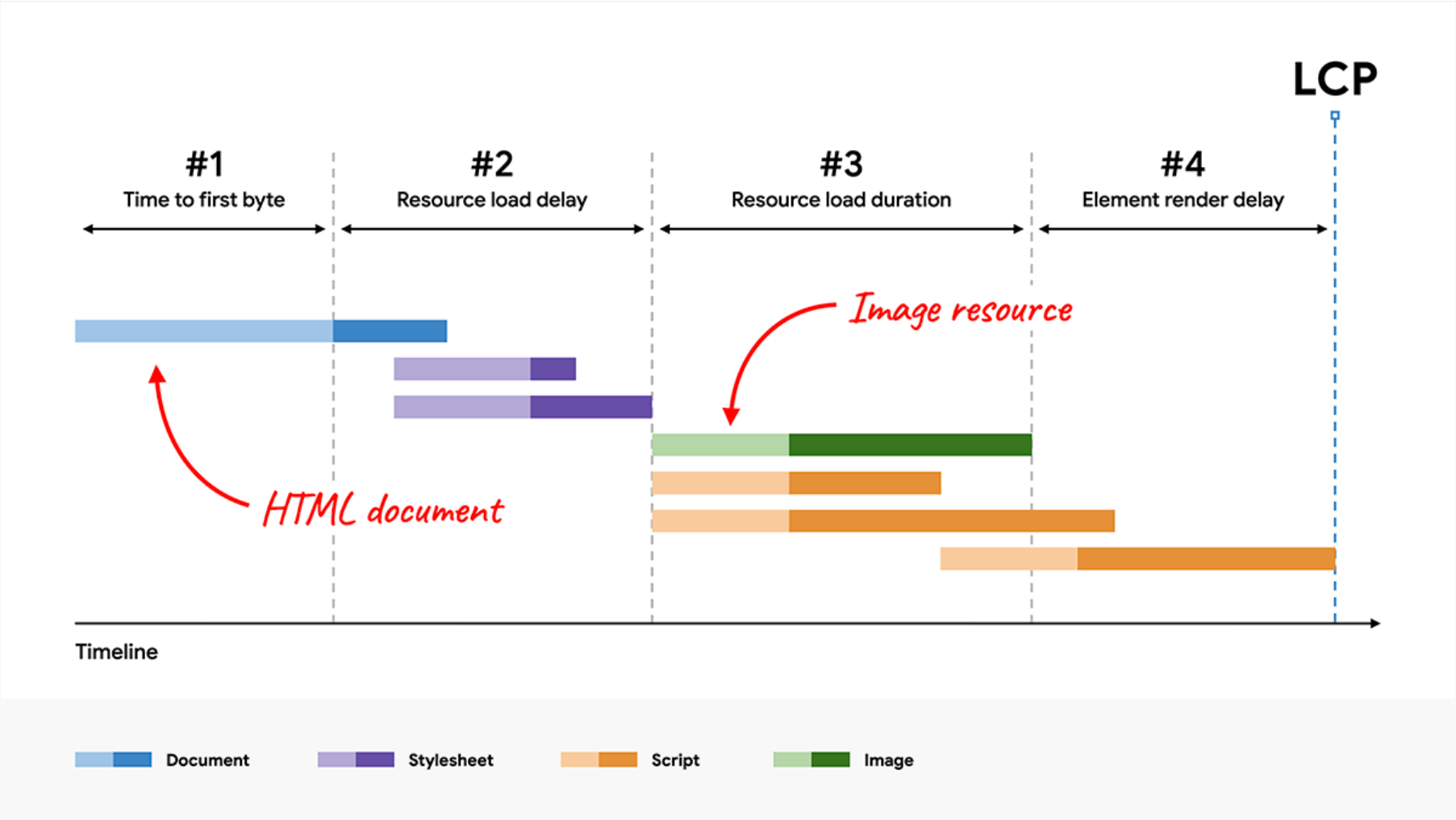
LCP: Largest Contentful Paint

These are measured in percentiles. You may look at 75th or 90th.

# First Contentful Paint



# Largest Contentful Paint



# Which is the Largest?

We might need to give the browser a hint what is the LCP.
    This is used by the preload scanner.

Preload scanner exists because the main parser can get blocked while working.
    Do things alongside this to look ahead and request what is needed.

Goal: prioritize the LCP element.

# Waterfall Loading

Waterfall: an unintentional sequence that should have been parallelized.

If we fetched data all at once we would save a lot of time.

Analogy: cooking without *mise en place*.



# Waterfall Loading

It's the sequential nature of requests that's the problem, not the number of them.

Rather like the $N + 1$ problem discussed much earlier, it is unlikely anyone sets out with the intention to write a waterfall.

It happens when there are nested components that depend on their parents.

# Do We Need It?

If the parent element *data* is needed, push this load into the backend.

If it's just the parent element *render*, then use the *mise en place* strategy: read the data tree first to find what to ask for.

# Rendering Pipeline

Once the data is loaded, the next part we need to be concerned about is how to actually render this efficiently.

If we assume that the user's display refreshes 60 times per second, a frame render needs to take place in about 10 ms, otherwise the page stutters.

This is sometimes referred to as "jank".

# Steps

Browser is mostly sequential, with limited parallelism.

1. JavaScript
2. Style
3. Layout
4. Paint
5. Composite

Goal: make the changes so that the browser starts at the latest stage possible.

# Shimmer

Consider a text shimmer example!

If the code changes background-position, this forces a repaint on every frame, so it requires more CPU time.

Animate a transform: the text is painted one time and the work is just composition: slide the overlay over a bit and stack the elements again.

That’s not for free, though, because the overlay needs to have its own layer, which takes up more memory.

# Cumulative Layout Shift

Sounds the same, but actually different: cumulative layout shift.

Perhaps you've had the experience on a webpage where you go to click on something and the page moves and you end up clicking the wrong thing?

This is a bad user experience, but it's not the focus of this section.

# Reduce Work

If big changes are not necessary, don’t do them.

We can think of the previous frame not only as the initial state, but also as cached results.

Minor changes invalidate a small part of it; major changes force more work.

# Layout Thrashing

Layout thrashing is the term for many forced layouts in quick succession.

The most efficient way for the browser to do the work it needs is to wait for all the changes to take place, then do the layout once.

If we don’t do that, we end up repeating a lot of work unnecessarily.

# Layout Thrashing

```
function resizeAllParagraphsToMatchBlockWidth () {
  // Puts the browser into a read–write–read–write cycle.
  for (let i = 0; i < paragraphs.length; i++) {
    paragraphs[i].style.width = `${box.offsetWidth}px`;
  }
}
```

```
// Read.
const width = box.offsetWidth;

function resizeAllParagraphsToMatchBlockWidth () {
  for (let i = 0; i < paragraphs.length; i++) {
    // Now write.
    paragraphs[i].style.width = `${width}px`;
  }
}
```

All of this describes the work required to render one frame!

A webpage has a Document Object Model (DOM) that represents the structure.

Google's Lighthouse performance tool considers the DOM to be a warning-level of complex at 800 nodes and an error-level of complex at 1400.

# Simplify the DOM

Simplifying the DOM is an obvious strategy: if there's less loaded on the page, there's less to do.

This may also result in less data being required from the backend, which then improves the data loading time. Two for one.

But if you need all the data on the page, it's hard to cut this down.

# Virtualization

If we can’t reduce the data, the plan is virtualization.

It means only creating elements for things that are visible on screen.

Imagine a calendar with hundreds of appointments; they can’t all fit on screen.

The tradeoff, though, is that elements that are virtualized are not in the DOM so we can’t find them with a search.

# Virtualization

Render them later?

There exists the `content-visibility` property in CSS that lets things be rendered on demand.

It reduces the work done on an initial page render and skips re-rendering for things that aren’t visible.

# How Much to Redraw?

If one data element is updated: risk that it forces a very large re-load/re-render.

Calendar example: change 1 appointment from 9:00 to 9:30.

If subscribed to the collection, the whole collection “changed”.
General tree comparison is $O(n^3)$ but even $O(n)$ is bad.

# False Sharing

Subscribing to the entire collection results in something that resembles false-sharing!

Appointments are in a lookup table by ID and there's a separate list of IDs?

The calendar component needs to look at the list of IDs and create one component per ID; that component gets its data from the table.

# What Makes for Good IDs?

If the data element itself has a unique ID (e.g., account number), good choice.

If we go by position in the list, it makes re-ordering a pain.

Random or unstable IDs are the worst choice because we could end up with a lot of unnecessary re-creation of elements.

# Not So Different

Get LCP done earlier and then call that “done” is a little bit unusual.

Everything else about frontend performance relates nicely to other things we’ve covered in the course!

Move things off the critical path, do less work, and trade space for time.